UniMentee ERP

Complete Full-Stack Implementation Guide

Stack	FastAPI · PostgreSQL (Supabase) · React 18 · TypeScript
Schema	University ERP v3.1 FINAL
PRD	Enhanced PRD v3.0 Enterprise Edition
Frontend Doc	Architecture v2.0
Backend Stages Done	0 – 3 (Setup, Auth, RBAC)
Remaining Stages	4 – 10 + Full Frontend
Date	March 2026

Follow this document stage-by-stage. Do not skip layers.

Section 1 — Current State & What to Build Next

You have completed the following stages from the backend guide. The table below shows exactly what exists and what needs to be built.

Stage	Name	Status	Output
0	Project setup, venv, pip install	✓ Done	unimentee-backend/ folder
1	Core infra: config, database, main	✓ Done	FastAPI server running
2	User + Auth module, JWT login	✓ Done	POST /auth/login works
3	RBAC + Multi-tenancy	✓ Done	university_id scoping
4	Academic Domain (subjects, offerings, curriculum)	Next	GET /subjects, /offerings APIs
5	Enrollment Engine (transactional)	Pending	POST /enrollments with locking
6	Mentor Module	Pending	Assignments + Sessions APIs
7	Attendance + Auto-Lock	Pending	Sessions, records, lock job
8	Marks + SGPA Engine	Pending	Marks entry, SGPA compute
9	Portfolio + Storage	Pending	Upload URLs, verification
10	Audit Logging Hardening	Pending	audit_logs append-only
F1	Frontend Foundation (React + Routing + Auth)	Pending	Login, RBAC, Layout shell
F2	Student, Faculty, Mentor Workspaces	Pending	Full dashboards + data entry
F3	Admin Suite + Analytics	Pending	Admin CRUD + reports

Important: Layer Rule

Every backend module follows: SQLAlchemy Model → Repository (DB queries) → Service (business logic) → Router (HTTP endpoints). Never put DB queries in routers. Never put business logic in repositories.

Section 2 — Backend Stages 4–10 (Complete Code)

Stage 4: Academic Domain → NEXT

This stage exposes the academic entities already in your database: subjects, programs, batches, sections, semesters, curriculum structures, and subject offerings. These are mostly read-heavy APIs with ACTIVE status filtering.

4.1 — Folder additions

Add these inside your existing app/ folder:

```
app/
  models/
    academic.py       # SQLAlchemy ORM models
  schemas/
    academic.py       # Pydantic request/response schemas
  repositories/
    academic_repository.py
  services/
    academic_service.py
  routers/
    academic_router.py
```

4.2 — models/academic.py

Map the key tables. You only need the fields your API exposes.

```
from sqlalchemy import Column, BigInteger, String, Boolean, Numeric, Integer, Date
from sqlalchemy import ForeignKey, Text
from sqlalchemy.dialects.postgresql import TIMESTAMPTZ
from app.database import Base


class Program(Base):
    __tablename__ = 'programs'
    program_id       = Column(BigInteger, primary_key=True)
    university_id    = Column(BigInteger, nullable=False)
    department_id    = Column(BigInteger)
    name             = Column(String(255), nullable=False)
    code             = Column(String(50), nullable=False)
    degree_type      = Column(String(50), nullable=False)
    duration_years   = Column(Numeric(3,1), nullable=False)
```

```
    total_semesters  = Column(Integer, nullable=False)
    status           = Column(String(20), default='ACTIVE')


class Batch(Base):
    __tablename__ = 'batches'
    batch_id      = Column(BigInteger, primary_key=True)
    university_id = Column(BigInteger, nullable=False)
    program_id    = Column(BigInteger, ForeignKey('programs.program_id'))
    batch_year    = Column(Integer, nullable=False)
    start_year    = Column(Integer, nullable=False)
    end_year      = Column(Integer, nullable=False)
    status        = Column(String(20), default='ACTIVE')


class Section(Base):
    __tablename__ = 'sections'
    section_id         = Column(BigInteger, primary_key=True)
    university_id      = Column(BigInteger, nullable=False)
    batch_id           = Column(BigInteger, ForeignKey('batches.batch_id'))
    name               = Column(String(10), nullable=False)
    capacity           = Column(Integer, default=60)
    current_strength   = Column(Integer, default=0)
    status             = Column(String(20), default='ACTIVE')


class Subject(Base):
    __tablename__ = 'subjects'
    subject_id    = Column(BigInteger, primary_key=True)
    university_id = Column(BigInteger, nullable=False)
    subject_code  = Column(String(50), nullable=False)
    subject_name  = Column(String(255), nullable=False)
    credits       = Column(Numeric(4,2), nullable=False)
    theory_hours  = Column(Numeric(4,2))
    lab_hours     = Column(Numeric(4,2))
    subject_type  = Column(String(20), default='THEORY')
    is_active     = Column(Boolean, default=True)


class SubjectOffering(Base):
    __tablename__ = 'subject_offerings'
    offering_id         = Column(BigInteger, primary_key=True)
```

```
    university_id       = Column(BigInteger, nullable=False)
    curriculum_id       = Column(BigInteger, nullable=False)
    batch_id            = Column(BigInteger, ForeignKey('batches.batch_id'))
    academic_year_id    = Column(BigInteger, nullable=False)
    term_id             = Column(BigInteger, nullable=False)
    section_id          = Column(BigInteger, ForeignKey('sections.section_id'))
    course_lead_id      = Column(BigInteger)
    status              = Column(String(20), default='DRAFT')
    current_enrollment  = Column(Integer, default=0)
    max_enrollment      = Column(Integer)
    version             = Column(Integer, default=1)
```

4.3 — schemas/academic.py

```
from pydantic import BaseModel
from typing import Optional


class ProgramOut(BaseModel):
    program_id: int
    name: str
    code: str
    degree_type: str
    duration_years: float
    total_semesters: int
    status: str
    class Config: from_attributes = True


class BatchOut(BaseModel):
    batch_id: int
    program_id: int
    batch_year: int
    status: str
    class Config: from_attributes = True


class SubjectOut(BaseModel):
    subject_id: int
    subject_code: str
    subject_name: str
```

```
    credits: float
    subject_type: str
    is_active: bool
    class Config: from_attributes = True


class OfferingOut(BaseModel):
    offering_id: int
    batch_id: int
    section_id: Optional[int]
    status: str
    current_enrollment: int
    version: int
    class Config: from_attributes = True


class OfferingStatusUpdate(BaseModel):
    status: str  # DRAFT | PLANNED | ACTIVE | COMPLETED | LOCKED | ARCHIVED
    version: int  # optimistic locking — send current version
```

4.4 — repositories/academic_repository.py

```
from sqlalchemy.orm import Session
from app.models.academic import Program, Batch, Section, Subject, SubjectOffering


def get_programs(db: Session, university_id: int):
    return db.query(Program).filter(
        Program.university_id == university_id,
        Program.status == 'ACTIVE'
    ).all()


def get_batches(db: Session, university_id: int, program_id: int = None):
    q = db.query(Batch).filter(Batch.university_id == university_id)
    if program_id:
        q = q.filter(Batch.program_id == program_id)
    return q.all()


def get_subjects(db: Session, university_id: int):
    return db.query(Subject).filter(
        Subject.university_id == university_id,
```

```
        Subject.is_active == True
    ).all()


def get_offerings(db: Session, university_id: int, term_id: int = None,
                  batch_id: int = None, section_id: int = None):
    q = db.query(SubjectOffering).filter(
        SubjectOffering.university_id == university_id
    )
    if term_id:    q = q.filter(SubjectOffering.term_id == term_id)
    if batch_id:   q = q.filter(SubjectOffering.batch_id == batch_id)
    if section_id: q = q.filter(SubjectOffering.section_id == section_id)
    return q.all()


def get_offering_by_id(db: Session, offering_id: int, university_id: int):
    return db.query(SubjectOffering).filter(
        SubjectOffering.offering_id == offering_id,
        SubjectOffering.university_id == university_id
    ).first()


def update_offering_status(db: Session, offering: SubjectOffering,
                           new_status: str, expected_version: int):
    if offering.version != expected_version:
        raise ValueError('Version conflict — reload and retry')
    offering.status = new_status
    # version auto-incremented by DB trigger
    db.commit()
    db.refresh(offering)
    return offering
```

4.5 — services/academic_service.py

```
from sqlalchemy.orm import Session
from app.repositories import academic_repository as repo


VALID_TRANSITIONS = {
    'DRAFT': ['PLANNED'],
    'PLANNED': ['ACTIVE'],
    'ACTIVE': ['COMPLETED', 'LOCKED'],
```

```
    'LOCKED': ['ACTIVE'],
    'COMPLETED': ['ARCHIVED'],
}


def list_programs(db: Session, university_id: int):
    return repo.get_programs(db, university_id)


def list_batches(db: Session, university_id: int, program_id: int = None):
    return repo.get_batches(db, university_id, program_id)


def list_subjects(db: Session, university_id: int):
    return repo.get_subjects(db, university_id)


def list_offerings(db: Session, university_id: int, term_id=None,
                   batch_id=None, section_id=None):
    return repo.get_offerings(db, university_id, term_id, batch_id, section_id)


def change_offering_status(db: Session, offering_id: int,
                           university_id: int, new_status: str, version: int):
    offering = repo.get_offering_by_id(db, offering_id, university_id)
    if not offering:
        raise LookupError('Offering not found')
    allowed = VALID_TRANSITIONS.get(offering.status, [])
    if new_status not in allowed:
        raise ValueError(f'Cannot move from {offering.status} to {new_status}')
    return repo.update_offering_status(db, offering, new_status, version)
```

4.6 — routers/academic_router.py

```
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from typing import Optional, List
from app.database import get_db
from app.core.rbac import get_current_user
from app.services import academic_service as svc
from app.schemas.academic import ProgramOut, BatchOut, SubjectOut, OfferingOut,
OfferingStatusUpdate


router = APIRouter(prefix='/academic', tags=['Academic'])
```

```
@router.get('/programs', response_model=List[ProgramOut])
def list_programs(user=Depends(get_current_user), db: Session = Depends(get_db)):
    return svc.list_programs(db, user['university_id'])

@router.get('/batches', response_model=List[BatchOut])
def list_batches(
    program_id: Optional[int] = Query(None),
    user=Depends(get_current_user), db: Session = Depends(get_db)):
    return svc.list_batches(db, user['university_id'], program_id)

@router.get('/subjects', response_model=List[SubjectOut])
def list_subjects(user=Depends(get_current_user), db: Session = Depends(get_db)):
    return svc.list_subjects(db, user['university_id'])

@router.get('/offerings', response_model=List[OfferingOut])
def list_offerings(
    term_id: Optional[int] = Query(None),
    batch_id: Optional[int] = Query(None),
    section_id: Optional[int] = Query(None),
    user=Depends(get_current_user), db: Session = Depends(get_db)):
    return svc.list_offerings(db, user['university_id'], term_id, batch_id, section_id)

@router.patch('/offerings/{offering_id}/status', response_model=OfferingOut)
def change_status(
    offering_id: int, body: OfferingStatusUpdate,
    user=Depends(get_current_user), db: Session = Depends(get_db)):
    try:
        return svc.change_offering_status(
            db, offering_id, user['university_id'], body.status, body.version)
    except LookupError as e: raise HTTPException(404, str(e))
    except ValueError as e:  raise HTTPException(409, str(e))
```

4.7 — Register in main.py

```
from app.routers import academic_router

app.include_router(academic_router.router)
```

Test it

GET /academic/programs → list of programs. GET /academic/offerings?term_id=1 → filter by term. PATCH /academic/offerings/1/status {status:'PLANNED', version:1} → status change.

Stage 5: Student Module + Enrollment Engine → NEXT

Students are the most complex entity. Enrollment must be transactional: it checks capacity, avoids duplicate enrollments, and updates current_enrollment atomically using SELECT FOR UPDATE to prevent race conditions.

5.1 — models/students.py

```
from sqlalchemy import Column, BigInteger, String, Date, Integer, Numeric, Boolean,
ForeignKey

from app.database import Base


class Student(Base):

    __tablename__ = 'students'

    student_id              = Column(BigInteger, primary_key=True)

    university_id           = Column(BigInteger, nullable=False)

    user_id                 = Column(BigInteger, ForeignKey('users.user_id'),
unique=True)

    usn                     = Column(String(50), nullable=False)

    program_id              = Column(BigInteger, nullable=False)

    batch_id                = Column(BigInteger, nullable=False)

    section_id              = Column(BigInteger)

    admission_date          = Column(Date, nullable=False)

    current_semester_number = Column(Integer)

    cgpa                    = Column(Numeric(4,2))

    status                  = Column(String(20), default='ACTIVE')

    version                 = Column(Integer, default=1)


class StudentSubjectEnrollment(Base):

    __tablename__ = 'student_subject_enrollments'

    enrollment_id   = Column(BigInteger, primary_key=True)

    university_id   = Column(BigInteger, nullable=False)

    student_id      = Column(BigInteger, ForeignKey('students.student_id'))

    offering_id     = Column(BigInteger, ForeignKey('subject_offerings.offering_id'))

    enrollment_type = Column(String(20), default='REGULAR')

    status          = Column(String(20), default='ENROLLED')

    version         = Column(Integer, default=1)
```

5.2 — repositories/student_repository.py

```
from sqlalchemy.orm import Session
```

```
from sqlalchemy import func
from app.models.students import Student, StudentSubjectEnrollment
from app.models.academic import SubjectOffering


def get_students(db: Session, university_id: int, batch_id=None,
                 section_id=None, status='ACTIVE'):
    q = db.query(Student).filter(Student.university_id == university_id)
    if batch_id:   q = q.filter(Student.batch_id == batch_id)
    if section_id: q = q.filter(Student.section_id == section_id)
    if status:     q = q.filter(Student.status == status)
    return q.all()


def get_student_by_id(db, student_id, university_id):
    return db.query(Student).filter(
        Student.student_id == student_id,
        Student.university_id == university_id).first()


def get_student_enrollments(db: Session, student_id: int):
    return db.query(StudentSubjectEnrollment).filter(
        StudentSubjectEnrollment.student_id == student_id,
        StudentSubjectEnrollment.status == 'ENROLLED'
    ).all()


def enroll_student_in_offering(
    db: Session, university_id: int, student_id: int, offering_id: int):
    # SELECT FOR UPDATE prevents race condition on capacity
    offering = db.query(SubjectOffering).with_for_update().filter(
        SubjectOffering.offering_id == offering_id,
        SubjectOffering.university_id == university_id
    ).first()
    if not offering:
        raise LookupError('Offering not found')
    if offering.status != 'ACTIVE':
        raise ValueError('Offering is not active')
    if offering.max_enrollment and offering.current_enrollment >=
offering.max_enrollment:
        raise ValueError('Offering is at full capacity')
    # Check existing enrollment
```

```
    existing = db.query(StudentSubjectEnrollment).filter(
        StudentSubjectEnrollment.student_id == student_id,
        StudentSubjectEnrollment.offering_id == offering_id
    ).first()
    if existing:
        raise ValueError('Student already enrolled')
    enrollment = StudentSubjectEnrollment(
        university_id=university_id,
        student_id=student_id,
        offering_id=offering_id,
        enrollment_type='REGULAR',
        status='ENROLLED'
    )
    db.add(enrollment)
    # DB trigger trg_sync_enrollment_count handles current_enrollment update
    db.commit()
    db.refresh(enrollment)
    return enrollment


def drop_enrollment(db: Session, enrollment_id: int, student_id: int):
    e = db.query(StudentSubjectEnrollment).filter(
        StudentSubjectEnrollment.enrollment_id == enrollment_id,
        StudentSubjectEnrollment.student_id == student_id
    ).first()
    if not e: raise LookupError('Enrollment not found')
    e.status = 'DROPPED'
    db.commit()
    return e
```

5.3 — schemas/students.py

```
from pydantic import BaseModel
from typing import Optional
from datetime import date


class StudentOut(BaseModel):
    student_id: int
    usn: str
    program_id: int
```

```
    batch_id: int
    section_id: Optional[int]
    current_semester_number: Optional[int]
    cgpa: Optional[float]
    status: str
    class Config: from_attributes = True


class EnrollmentIn(BaseModel):
    offering_id: int


class EnrollmentOut(BaseModel):
    enrollment_id: int
    student_id: int
    offering_id: int
    enrollment_type: str
    status: str
    class Config: from_attributes = True
```

5.4 — services/student_service.py

```
from sqlalchemy.orm import Session
from app.repositories import student_repository as repo


def list_students(db, university_id, batch_id=None, section_id=None):
    return repo.get_students(db, university_id, batch_id, section_id)


def get_student(db, student_id, university_id):
    s = repo.get_student_by_id(db, student_id, university_id)
    if not s: raise LookupError('Student not found')
    return s


def get_student_enrollments(db, student_id, university_id):
    get_student(db, student_id, university_id)  # verify access
    return repo.get_student_enrollments(db, student_id)


def enroll(db, university_id, student_id, offering_id):
    try:
        return repo.enroll_student_in_offering(db, university_id, student_id,
offering_id)
```

```
    except Exception:
        db.rollback()
        raise


def drop(db, enrollment_id, student_id):
    return repo.drop_enrollment(db, enrollment_id, student_id)
```

5.5 — routers/student_router.py

```
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from typing import Optional, List
from app.database import get_db
from app.core.rbac import get_current_user
from app.services import student_service as svc
from app.schemas.students import StudentOut, EnrollmentIn, EnrollmentOut


router = APIRouter(prefix='/students', tags=['Students'])


@router.get('', response_model=List[StudentOut])
def list_students(
    batch_id: Optional[int] = Query(None),
    section_id: Optional[int] = Query(None),
    user=Depends(get_current_user), db: Session = Depends(get_db)):
    return svc.list_students(db, user['university_id'], batch_id, section_id)


@router.get('/{student_id}', response_model=StudentOut)
def get_student(
    student_id: int,
    user=Depends(get_current_user), db: Session = Depends(get_db)):
    try: return svc.get_student(db, student_id, user['university_id'])
    except LookupError as e: raise HTTPException(404, str(e))


@router.get('/{student_id}/enrollments', response_model=List[EnrollmentOut])
def get_enrollments(
    student_id: int,
    user=Depends(get_current_user), db: Session = Depends(get_db)):
    return svc.get_student_enrollments(db, student_id, user['university_id'])
```

```
@router.post('/{student_id}/enrollments', response_model=EnrollmentOut)
def enroll(
    student_id: int, body: EnrollmentIn,
    user=Depends(get_current_user), db: Session = Depends(get_db)):
    try:
        return svc.enroll(db, user['university_id'], student_id, body.offering_id)
    except LookupError as e: raise HTTPException(404, str(e))
    except ValueError as e:  raise HTTPException(409, str(e))

@router.delete('/enrollments/{enrollment_id}')
def drop(
    enrollment_id: int,
    user=Depends(get_current_user), db: Session = Depends(get_db)):
    try: return svc.drop(db, enrollment_id, user['user_id'])
    except LookupError as e: raise HTTPException(404, str(e))
```

Register in main.py
from app.routers import student_router app.include_router(student_router.router)

Stage 6: Mentor Module

→ NEXT

The mentor module manages assignments (which mentor is assigned to which student) and mentoring sessions (logged meetings). The DB already has the unique partial index ensuring only one ACTIVE assignment per student.

6.1 — models/mentorship.py

```
from sqlalchemy import Column, BigInteger, String, Date, Time, Integer, Boolean, Text,
ForeignKey

from sqlalchemy.dialects.postgresql import TIMESTAMPTZ

from app.database import Base


class MentorAssignment(Base):

    __tablename__ = 'mentor_assignments'

    assignment_id    = Column(BigInteger, primary_key=True)

    university_id    = Column(BigInteger, nullable=False)

    student_id       = Column(BigInteger, ForeignKey('students.student_id'))

    mentor_user_id   = Column(BigInteger, ForeignKey('users.user_id'))

    academic_year_id = Column(BigInteger, nullable=False)

    assigned_by      = Column(BigInteger)

    status           = Column(String(20), default='ACTIVE')

    version          = Column(Integer, default=1)


class MentoringSession(Base):

    __tablename__ = 'mentoring_sessions'

    session_id          = Column(BigInteger, primary_key=True)

    university_id       = Column(BigInteger, nullable=False)

    assignment_id       = Column(BigInteger,
ForeignKey('mentor_assignments.assignment_id'))

    session_date        = Column(Date, nullable=False)

    session_time        = Column(Time)

    duration_minutes    = Column(Integer)

    session_type        = Column(String(20), default='IN_PERSON')

    topics_discussed    = Column(Text)

    action_items        = Column(Text)

    follow_up_required  = Column(Boolean, default=False)

    follow_up_date      = Column(Date)

    career_notes        = Column(Text)

    created_by          = Column(BigInteger, ForeignKey('users.user_id'))
```

6.2 — schemas/mentorship.py

```
from pydantic import BaseModel
from typing import Optional
from datetime import date, time


class AssignmentOut(BaseModel):
    assignment_id: int
    student_id: int
    mentor_user_id: int
    status: str
    class Config: from_attributes = True


class SessionIn(BaseModel):
    session_date: date
    session_time: Optional[time]
    duration_minutes: Optional[int]
    session_type: str = 'IN_PERSON'
    topics_discussed: Optional[str]
    action_items: Optional[str]
    follow_up_required: bool = False
    follow_up_date: Optional[date]
    career_notes: Optional[str]


class SessionOut(SessionIn):
    session_id: int
    assignment_id: int
    created_by: int
    class Config: from_attributes = True
```

6.3 — repositories/mentor_repository.py

```
from sqlalchemy.orm import Session
from app.models.mentorship import MentorAssignment, MentoringSession


def get_assignments_for_mentor(db: Session, mentor_user_id: int, university_id: int):
    return db.query(MentorAssignment).filter(
        MentorAssignment.mentor_user_id == mentor_user_id,
```

```
        MentorAssignment.university_id == university_id,
        MentorAssignment.status == 'ACTIVE'
    ).all()

def get_assignment_for_student(db: Session, student_id: int, university_id: int):
    return db.query(MentorAssignment).filter(
        MentorAssignment.student_id == student_id,
        MentorAssignment.university_id == university_id,
        MentorAssignment.status == 'ACTIVE'
    ).first()

def get_sessions(db: Session, assignment_id: int):
    return db.query(MentoringSession).filter(
        MentoringSession.assignment_id == assignment_id
    ).order_by(MentoringSession.session_date.desc()).all()

def create_session(db: Session, assignment_id: int, university_id: int,
                   created_by: int, data: dict):
    session = MentoringSession(
        university_id=university_id,
        assignment_id=assignment_id,
        created_by=created_by,
        **data
    )
    db.add(session)
    db.commit()
    db.refresh(session)
    return session
```

6.4 — routers/mentor_router.py

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import get_db
from app.core.rbac import get_current_user
from app.repositories import mentor_repository as repo
from app.schemas.mentorship import AssignmentOut, SessionIn, SessionOut
from typing import List
```

```
router = APIRouter(prefix='/mentor', tags=['Mentor'])


@router.get('/assignments', response_model=List[AssignmentOut])
def my_assignments(user=Depends(get_current_user), db=Depends(get_db)):
    return repo.get_assignments_for_mentor(db, user['user_id'], user['university_id'])


@router.get('/assignments/{assignment_id}/sessions', response_model=List[SessionOut])
def get_sessions(assignment_id: int, user=Depends(get_current_user),
db=Depends(get_db)):
    return repo.get_sessions(db, assignment_id)


@router.post('/assignments/{assignment_id}/sessions', response_model=SessionOut)
def create_session(
    assignment_id: int, body: SessionIn,
    user=Depends(get_current_user), db=Depends(get_db)):
    return repo.create_session(
        db, assignment_id, user['university_id'],
        user['user_id'], body.model_dump(exclude_none=True))


# Student: view own mentor sessions
@router.get('/my-sessions', response_model=List[SessionOut])
def my_sessions(user=Depends(get_current_user), db=Depends(get_db)):
    assignment = repo.get_assignment_for_student(
        db, user['user_id'], user['university_id'])
    if not assignment: raise HTTPException(404, 'No active mentor assignment')
    return repo.get_sessions(db, assignment.assignment_id)
```

Stage 7: Attendance Module + Auto-Lock → NEXT

Attendance is high-frequency. Sessions are created per class. Records are bulk-updated. The DB has UNIQUE(offering_id, session_date, start_time) — return 409 on duplicate. Auto-lock runs after university_settings.attendance_auto_lock_hours via a background task.

7.1 — models/attendance.py

```
from sqlalchemy import Column, BigInteger, String, Date, Time, Integer, Boolean, Text, ForeignKey
from sqlalchemy.dialects.postgresql import TIMESTAMPTZ
from app.database import Base


class AttendanceSession(Base):
    __tablename__ = 'attendance_sessions'
    session_id    = Column(BigInteger, primary_key=True)
    university_id = Column(BigInteger, nullable=False)
    offering_id   = Column(BigInteger, ForeignKey('subject_offerings.offering_id'))
    session_date  = Column(Date, nullable=False)
    start_time    = Column(Time, nullable=False)
    end_time      = Column(Time, nullable=False)
    session_type  = Column(String(20), default='THEORY')
    topic_covered = Column(Text)
    conducted_by  = Column(BigInteger, ForeignKey('users.user_id'))
    total_present = Column(Integer)
    is_locked     = Column(Boolean, default=False)
    locked_at     = Column(TIMESTAMPTZ)
    locked_by     = Column(String(20))


class AttendanceRecord(Base):
    __tablename__ = 'attendance_records'
    attendance_id = Column(BigInteger, primary_key=True)
    university_id = Column(BigInteger, nullable=False)
    session_id    = Column(BigInteger, ForeignKey('attendance_sessions.session_id'))
    student_id    = Column(BigInteger, ForeignKey('students.student_id'))
    status        = Column(String(20), default='ABSENT')
    marked_at     = Column(TIMESTAMPTZ)
    note          = Column(Text)
```

7.2 — schemas/attendance.py

```
from pydantic import BaseModel
from typing import Optional, List
from datetime import date, time


class SessionIn(BaseModel):
    session_date: date
    start_time: time
    end_time: time
    session_type: str = 'THEORY'
    topic_covered: Optional[str]


class SessionOut(SessionIn):
    session_id: int
    offering_id: int
    is_locked: bool
    total_present: Optional[int]
    class Config: from_attributes = True


class AttendanceRecordIn(BaseModel):
    student_id: int
    status: str  # PRESENT | ABSENT | LATE | ON_LEAVE


class BulkAttendanceIn(BaseModel):
    records: List[AttendanceRecordIn]
```

7.3 — repositories/attendance_repository.py

```
from sqlalchemy.orm import Session
from sqlalchemy.dialects.postgresql import insert as pg_insert
from app.models.attendance import AttendanceSession, AttendanceRecord
from datetime import datetime, timezone


def create_session(db: Session, university_id: int, offering_id: int,
                   conducted_by: int, data: dict):
    s = AttendanceSession(
        university_id=university_id,
        offering_id=offering_id,
```

```
        conducted_by=conducted_by,
        **data
    )
    db.add(s)
    try:
        db.commit()
    except Exception:
        db.rollback()
        raise  # 409 handled by router catching IntegrityError
    db.refresh(s)
    return s


def get_sessions(db: Session, offering_id: int):
    return db.query(AttendanceSession).filter(
        AttendanceSession.offering_id == offering_id
    ).order_by(AttendanceSession.session_date.desc()).all()


def get_session(db, session_id, university_id):
    return db.query(AttendanceSession).filter(
        AttendanceSession.session_id == session_id,
        AttendanceSession.university_id == university_id
    ).first()


def bulk_upsert_records(db: Session, session_id: int,
                        university_id: int, records: list):
    for r in records:
        stmt = pg_insert(AttendanceRecord).values(
            session_id=session_id, university_id=university_id,
            student_id=r['student_id'], status=r['status'],
            marked_at=datetime.now(timezone.utc)
        ).on_conflict_do_update(
            index_elements=['session_id', 'student_id'],
            set_={'status': r['status'], 'marked_at': datetime.now(timezone.utc)}
        )
        db.execute(stmt)
    # Update total_present count
    present_count = sum(1 for r in records if r['status'] in ('PRESENT','LATE'))
    db.query(AttendanceSession).filter(
```

```
        AttendanceSession.session_id == session_id
    ).update({'total_present': present_count})
    db.commit()


def lock_session(db: Session, session: AttendanceSession, by: str = 'MANUAL'):
    session.is_locked = True
    session.locked_at = datetime.now(timezone.utc)
    session.locked_by = by
    db.commit()
    db.refresh(session)
    return session


def auto_lock_expired(db: Session, university_id: int, lock_after_hours: int):
    """Lock all sessions older than lock_after_hours that are not yet locked."""
    from datetime import timedelta
    cutoff = datetime.now(timezone.utc) - timedelta(hours=lock_after_hours)
    sessions = db.query(AttendanceSession).filter(
        AttendanceSession.university_id == university_id,
        AttendanceSession.is_locked == False,
        AttendanceSession.locked_at == None
    ).all()
    for s in sessions:
        if s.session_date and str(s.session_date) < cutoff.strftime('%Y-%m-%d'):
            lock_session(db, s, by='AUTO')
```

7.4 — routers/attendance_router.py

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from sqlalchemy.exc import IntegrityError
from app.database import get_db
from app.core.rbac import get_current_user
from app.repositories import attendance_repository as repo
from app.schemas.attendance import SessionIn, SessionOut, BulkAttendanceIn
from typing import List


router = APIRouter(tags=['Attendance'])
```

```
@router.post('/offerings/{offering_id}/sessions', response_model=SessionOut)
def create_session(
    offering_id: int, body: SessionIn,
    user=Depends(get_current_user), db=Depends(get_db)):
    try:
        return repo.create_session(
            db, user['university_id'], offering_id,
            user['user_id'], body.model_dump(exclude_none=True))
    except IntegrityError:
        raise HTTPException(409, 'Session already exists for this time slot')

@router.get('/offerings/{offering_id}/sessions', response_model=List[SessionOut])
def list_sessions(
    offering_id: int,
    user=Depends(get_current_user), db=Depends(get_db)):
    return repo.get_sessions(db, offering_id)

@router.put('/sessions/{session_id}/attendance')
def bulk_mark(
    session_id: int, body: BulkAttendanceIn,
    user=Depends(get_current_user), db=Depends(get_db)):
    session = repo.get_session(db, session_id, user['university_id'])
    if not session: raise HTTPException(404, 'Session not found')
    if session.is_locked: raise HTTPException(403, 'Session is locked')
    repo.bulk_upsert_records(
        db, session_id, user['university_id'],
        [r.model_dump() for r in body.records])
    return {'message': 'Attendance saved'}

@router.patch('/sessions/{session_id}/lock')
def lock(
    session_id: int,
    user=Depends(get_current_user), db=Depends(get_db)):
    session = repo.get_session(db, session_id, user['university_id'])
    if not session: raise HTTPException(404)
    return repo.lock_session(db, session)
```

7.5 — Background Auto-Lock (add to main.py)

Add a startup background task to periodically auto-lock old sessions.

```
from fastapi import FastAPI
from contextlib import asynccontextmanager
import asyncio


async def auto_lock_loop():
    from app.database import SessionLocal
    from app.repositories.attendance_repository import auto_lock_expired
    while True:
        await asyncio.sleep(3600)  # run every hour
        db = SessionLocal()
        try:
            # lock_after_hours=24 (from university_settings default)
            auto_lock_expired(db, university_id=1, lock_after_hours=24)
        finally:
            db.close()


@asynccontextmanager
async def lifespan(app):
    asyncio.create_task(auto_lock_loop())
    yield


app = FastAPI(title='UniMentee ERP API', lifespan=lifespan)
```

Stage 8: Marks + SGPA Engine → NEXT

Marks entry is version-controlled (optimistic locking). The DB trigger check_marks_max enforces marks <= max_marks. When an offering is marked COMPLETED, compute SGPA for all enrolled students and update student_academic_progress.

8.1 — models/marks.py

```
from sqlalchemy import Column, BigInteger, String, Numeric, Boolean, Integer, Text,
Date

from sqlalchemy import ForeignKey

from sqlalchemy.dialects.postgresql import TIMESTAMPTZ

from app.database import Base


class AssessmentType(Base):

    __tablename__ = 'assessment_types'

    assessment_type_id = Column(BigInteger, primary_key=True)

    university_id      = Column(BigInteger, nullable=False)

    name               = Column(String(100), nullable=False)

    code               = Column(String(20), nullable=False)

    weightage          = Column(Numeric(5,2))

    is_internal        = Column(Boolean, default=True)


class Assessment(Base):

    __tablename__ = 'assessments'

    assessment_id      = Column(BigInteger, primary_key=True)

    university_id      = Column(BigInteger, nullable=False)

    offering_id        = Column(BigInteger,
ForeignKey('subject_offerings.offering_id'))

    assessment_type_id = Column(BigInteger,
ForeignKey('assessment_types.assessment_type_id'))

    title              = Column(String(255))

    max_marks          = Column(Numeric(6,2), nullable=False)

    passing_marks      = Column(Numeric(6,2))

    conducted_on       = Column(Date)

    status             = Column(String(20), default='DRAFT')

    submitted_by       = Column(BigInteger)

    verified_by        = Column(BigInteger)

    published_by       = Column(BigInteger)

    version            = Column(Integer, default=1)
```

```
class StudentMark(Base):
    __tablename__ = 'student_marks'
    mark_id        = Column(BigInteger, primary_key=True)
    university_id  = Column(BigInteger, nullable=False)
    assessment_id  = Column(BigInteger, ForeignKey('assessments.assessment_id'))
    student_id     = Column(BigInteger, ForeignKey('students.student_id'))
    marks_obtained = Column(Numeric(6,2))
    is_absent      = Column(Boolean, default=False)
    remark         = Column(Text)
    entered_by     = Column(BigInteger, ForeignKey('users.user_id'))
    version        = Column(Integer, default=1)


class StudentAcademicProgress(Base):
    __tablename__ = 'student_academic_progress'
    progress_id      = Column(BigInteger, primary_key=True)
    student_id       = Column(BigInteger, ForeignKey('students.student_id'))
    academic_year_id = Column(BigInteger, nullable=False)
    term_id          = Column(BigInteger, nullable=False)
    semester_number  = Column(Integer, nullable=False)
    sgpa             = Column(Numeric(4,2))
    cgpa             = Column(Numeric(4,2))
    sgpa_status      = Column(String(20), default='PENDING')


class GradeScale(Base):
    __tablename__ = 'grade_scales'
    grade_scale_id = Column(BigInteger, primary_key=True)
    university_id  = Column(BigInteger, nullable=False)
    grade_letter   = Column(String(5), nullable=False)
    grade_point    = Column(Numeric(4,2), nullable=False)
    min_percentage = Column(Numeric(5,2), nullable=False)
    max_percentage = Column(Numeric(5,2), nullable=False)
    is_passing     = Column(Boolean, default=True)
```

8.2 — services/marks_service.py (with SGPA computation)

```
from sqlalchemy.orm import Session
from sqlalchemy.dialects.postgresql import insert as pg_insert
```

```
from app.models.marks import Assessment, StudentMark, StudentAcademicProgress,
GradeScale
from app.models.students import StudentSubjectEnrollment, Student
from app.models.academic import SubjectOffering
from app.repositories import academic_repository as academic_repo


MARK_STATUS_TRANSITIONS = {
    'DRAFT':     'SUBMITTED',
    'SUBMITTED': 'VERIFIED',
    'VERIFIED':  'PUBLISHED',
}


def get_assessments(db: Session, offering_id: int):
    return db.query(Assessment).filter(
        Assessment.offering_id == offering_id).all()


def get_marks(db: Session, assessment_id: int):
    return db.query(StudentMark).filter(
        StudentMark.assessment_id == assessment_id).all()


def upsert_mark(db: Session, university_id: int, assessment_id: int,
                student_id: int, marks_obtained, is_absent: bool,
                entered_by: int, current_version: int):
    """Upsert a single mark. The DB trigger validates marks <= max_marks."""
    existing = db.query(StudentMark).filter(
        StudentMark.assessment_id == assessment_id,
        StudentMark.student_id == student_id
    ).first()
    if existing:
        if existing.version != current_version:
            raise ValueError('Version conflict — reload and retry')
        existing.marks_obtained = marks_obtained
        existing.is_absent = is_absent
        existing.entered_by = entered_by
    else:
        existing = StudentMark(
            university_id=university_id,
            assessment_id=assessment_id,
```

```
            student_id=student_id,
            marks_obtained=marks_obtained,
            is_absent=is_absent,
            entered_by=entered_by
        )
        db.add(existing)
    try:
        db.commit()
    except Exception:
        db.rollback()
        raise  # DB trigger raises if marks > max_marks
    db.refresh(existing)
    return existing


def advance_assessment_status(db: Session, assessment_id: int,
                              university_id: int, actor_id: int):
    a = db.query(Assessment).filter(
        Assessment.assessment_id == assessment_id,
        Assessment.university_id == university_id
    ).first()
    if not a: raise LookupError('Assessment not found')
    next_status = MARK_STATUS_TRANSITIONS.get(a.status)
    if not next_status: raise ValueError(f'Cannot advance from {a.status}')
    a.status = next_status
    if next_status == 'SUBMITTED': a.submitted_by = actor_id
    if next_status == 'VERIFIED':  a.verified_by  = actor_id
    if next_status == 'PUBLISHED':
        a.published_by = actor_id
        compute_sgpa_for_offering(db, a.offering_id, university_id)
    db.commit()
    return a


def compute_sgpa_for_offering(db: Session, offering_id: int, university_id: int):
    """Called when last assessment for offering is published. Computes SGPA."""
    offering = db.query(SubjectOffering).filter(
        SubjectOffering.offering_id == offering_id).first()
    grade_scales = db.query(GradeScale).filter(
        GradeScale.university_id == university_id
```

```
    ).order_by(GradeScale.min_percentage.desc()).all()
    enrollments = db.query(StudentSubjectEnrollment).filter(
        StudentSubjectEnrollment.offering_id == offering_id,
        StudentSubjectEnrollment.status == 'ENROLLED'
    ).all()
    for enrollment in enrollments:
        marks = db.query(StudentMark).filter(
            StudentMark.assessment_id.in_(
                db.query(Assessment.assessment_id).filter(
                    Assessment.offering_id == offering_id,
                    Assessment.status == 'PUBLISHED'
                ).subquery()
            ),
            StudentMark.student_id == enrollment.student_id
        ).all()
        if not marks: continue
        # Compute weighted % from IA1, IA2, SEE etc
        total = sum(float(m.marks_obtained or 0) for m in marks)
        max_total = sum(
            float(db.query(Assessment.max_marks).filter(
                Assessment.assessment_id == m.assessment_id).scalar() or 0)
            for m in marks
        )
        pct = (total / max_total * 100) if max_total > 0 else 0
        # Map pct to grade point
        grade_point = 0.0
        for gs in grade_scales:
            if pct >= float(gs.min_percentage):
                grade_point = float(gs.grade_point)
                break
        # Simple SGPA = average grade point across enrolled subjects
        # (Full SGPA uses credits — simplified here for clarity)
        student = db.query(Student).filter(
            Student.student_id == enrollment.student_id).first()
        if student:
            # Update or insert academic progress
            prog = db.query(StudentAcademicProgress).filter(
                StudentAcademicProgress.student_id == enrollment.student_id,
```

```
                StudentAcademicProgress.term_id == offering.term_id
            ).first()
            if not prog:
                prog = StudentAcademicProgress(
                    student_id=enrollment.student_id,
                    academic_year_id=offering.academic_year_id,
                    term_id=offering.term_id,
                    semester_number=student.current_semester_number or 1
                )
                db.add(prog)
            prog.sgpa_status = 'COMPUTED'
            db.commit()
```

8.3 — routers/marks_router.py

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from pydantic import BaseModel
from typing import Optional, List
from app.database import get_db
from app.core.rbac import get_current_user
from app.services import marks_service as svc

router = APIRouter(tags=['Marks'])

class MarkIn(BaseModel):
    student_id: int
    marks_obtained: Optional[float]
    is_absent: bool = False
    version: int = 1

class MarkOut(BaseModel):
    mark_id: int
    assessment_id: int
    student_id: int
    marks_obtained: Optional[float]
    is_absent: bool
    version: int
```

```
    class Config: from_attributes = True


@router.get('/offerings/{offering_id}/assessments')
def list_assessments(
    offering_id: int,
    user=Depends(get_current_user), db=Depends(get_db)):
    return svc.get_assessments(db, offering_id)


@router.get('/assessments/{assessment_id}/marks', response_model=List[MarkOut])
def list_marks(
    assessment_id: int,
    user=Depends(get_current_user), db=Depends(get_db)):
    return svc.get_marks(db, assessment_id)


@router.put('/assessments/{assessment_id}/marks/{student_id}', response_model=MarkOut)
def upsert_mark(
    assessment_id: int, student_id: int, body: MarkIn,
    user=Depends(get_current_user), db=Depends(get_db)):
    try:
        return svc.upsert_mark(
            db, user['university_id'], assessment_id,
            student_id, body.marks_obtained, body.is_absent,
            user['user_id'], body.version)
    except ValueError as e: raise HTTPException(409, str(e))
    except Exception as e:   raise HTTPException(400, str(e))


@router.patch('/assessments/{assessment_id}/status')
def advance_status(
    assessment_id: int,
    user=Depends(get_current_user), db=Depends(get_db)):
    try:
        return svc.advance_assessment_status(
            db, assessment_id, user['university_id'], user['user_id'])
    except LookupError as e: raise HTTPException(404, str(e))
    except ValueError as e:  raise HTTPException(400, str(e))
```

Stage 9: Portfolio + Storage + Audit Hardening → NEXT

Portfolio items store document metadata only — never binary data. Files go to Supabase Storage (S3-compatible). Presigned upload URLs are generated server-side. Audit logging wraps every mutating service call.

9.1 — models/portfolio.py

```
from sqlalchemy import Column, BigInteger, String, Date, Boolean, Text, Integer
from sqlalchemy import ForeignKey
from sqlalchemy.dialects.postgresql import TIMESTAMPTZ
from app.database import Base
import uuid


class PortfolioItem(Base):
    __tablename__ = 'portfolio_items'
    item_id           = Column(BigInteger, primary_key=True)
    student_id        = Column(BigInteger, ForeignKey('students.student_id'))
    item_type         = Column(String(30), nullable=False)
    title             = Column(String(255), nullable=False)
    issuing_org       = Column(String(255))
    issue_date        = Column(Date, nullable=False)
    description       = Column(Text)
    file_url          = Column(String(500), nullable=False)
    file_key          = Column(String(500), nullable=False)
    uploaded_by       = Column(BigInteger, ForeignKey('users.user_id'))
    verification_code = Column(String(100), nullable=False)
    status            = Column(String(20), default='PENDING')
    verified_by       = Column(BigInteger)
    version           = Column(Integer, default=1)
```

9.2 — Storage Service (Supabase)

Install: pip install supabase --break-system-packages

```
# services/storage_service.py
import os
from supabase import create_client


SUPABASE_URL = os.getenv('SUPABASE_URL')
SUPABASE_KEY = os.getenv('SUPABASE_SERVICE_KEY')  # service role key
```

```
BUCKET = 'portfolio-documents'


supabase = create_client(SUPABASE_URL, SUPABASE_KEY)


def generate_upload_url(file_key: str, content_type: str = 'application/pdf'):
    """
    Returns a presigned URL for direct client-side upload.
    file_key example: 'portfolios/student_42/cert_abc.pdf'
    """
    response = supabase.storage.from_(BUCKET).create_signed_upload_url(file_key)
    return {
        'upload_url': response['signedUrl'],
        'file_key': file_key,
        'public_url': f'{SUPABASE_URL}/storage/v1/object/public/{BUCKET}/{file_key}'
    }


def get_public_url(file_key: str):
    return supabase.storage.from_(BUCKET).get_public_url(file_key)
```

9.3 — Audit Service

```
# services/audit_service.py
from sqlalchemy.orm import Session
from sqlalchemy import text
import json


def log_action(db: Session, university_id: int, actor_user_id: int,
               action: str, entity_type: str, entity_id: int,
               old_value: dict = None, new_value: dict = None,
               ip_address: str = None):
    """
    Insert into audit_logs. This table is APPEND-ONLY (REVOKE UPDATE/DELETE in DB).
    Call this from service layer only — never from routers or repositories.
    """
    db.execute(text(
        """INSERT INTO audit_logs
           (university_id, actor_user_id, action, entity_type, entity_id,
            old_value, new_value, ip_address)
```

```
            VALUES (:uid, :actor, :action, :etype, :eid, :old, :new, :ip)"""
    ), {
        'uid': university_id,
        'actor': actor_user_id,
        'action': action,
        'etype': entity_type,
        'eid': entity_id,
        'old': json.dumps(old_value) if old_value else None,
        'new': json.dumps(new_value) if new_value else None,
        'ip': ip_address
    })
    # Note: do NOT call db.commit() here — let the calling service commit
```

9.4 — Wiring Audit into Marks Service (example)

Wrap every important mutation in audit_service.log_action:

```
# In marks_service.py — advance_assessment_status
from app.services import audit_service


def advance_assessment_status(db, assessment_id, university_id, actor_id):
    a = ...  # existing code to get assessment
    old_status = a.status
    # ... transition logic ...
    audit_service.log_action(
        db, university_id, actor_id,
        action='UPDATE',
        entity_type='assessments',
        entity_id=assessment_id,
        old_value={'status': old_status},
        new_value={'status': a.status}
    )
    db.commit()
    return a
```

9.5 — Portfolio Router

```
# routers/portfolio_router.py
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
```

```
from pydantic import BaseModel
from typing import Optional
from datetime import date
from app.database import get_db
from app.core.rbac import get_current_user
from app.services.storage_service import generate_upload_url
from app.models.portfolio import PortfolioItem
import uuid

router = APIRouter(prefix='/portfolio', tags=['Portfolio'])

class UploadRequest(BaseModel):
    filename: str
    content_type: str = 'application/pdf'

class PortfolioItemIn(BaseModel):
    item_type: str
    title: str
    issuing_org: Optional[str]
    issue_date: date
    description: Optional[str]
    file_key: str  # from the upload URL response
    file_url: str

@router.post('/upload-url')
def get_upload_url(
    body: UploadRequest,
    user=Depends(get_current_user)):
    file_key = f'portfolios/student_{user["user_id"]}/{uuid.uuid4()}_{body.filename}'
    return generate_upload_url(file_key, body.content_type)

@router.post('/items')
def create_item(
    body: PortfolioItemIn,
    user=Depends(get_current_user), db=Depends(get_db)):
    item = PortfolioItem(
        student_id=user['user_id'],  # resolve student_id from user_id in prod
        uploaded_by=user['user_id'],
```

```
        verification_code=str(uuid.uuid4()),
        **body.model_dump()
    )
    db.add(item)
    db.commit()
    return {'item_id': item.item_id, 'verification_code': item.verification_code}


@router.get('/items')
def list_items(user=Depends(get_current_user), db=Depends(get_db)):
    return db.query(PortfolioItem).filter(
        PortfolioItem.student_id == user['user_id']).all()
```

Section 3 — Complete main.py (All Stages Registered)

Replace your existing main.py with this. Every router is registered here.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
import asyncio


from app.routers import (
    academic_router,
    student_router,
    mentor_router,
    attendance_router,
    marks_router,
    portfolio_router,
)
from app.routers.auth import router as auth_router


async def auto_lock_loop():
    from app.database import SessionLocal
    from app.repositories.attendance_repository import auto_lock_expired
    while True:
        await asyncio.sleep(3600)
        db = SessionLocal()
        try:
            auto_lock_expired(db, university_id=1, lock_after_hours=24)
        finally:
            db.close()


@asynccontextmanager
async def lifespan(app):
    asyncio.create_task(auto_lock_loop())
    yield


app = FastAPI(title='UniMentee ERP API', version='1.0.0', lifespan=lifespan)


app.add_middleware(
    CORSMiddleware,
```

```
    allow_origins=['http://localhost:5173'],  # Vite dev server
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)


app.include_router(auth_router)
app.include_router(academic_router.router)
app.include_router(student_router.router)
app.include_router(mentor_router.router)
app.include_router(attendance_router.router)
app.include_router(marks_router.router)
app.include_router(portfolio_router.router)


@app.get('/')
def root():
    return {'status': 'UniMentee ERP API running', 'docs': '/docs'}
```

Section 4 — Frontend Setup (React 18 + TypeScript)

4.1 — Project Creation

```
npm create vite@latest unimentee-frontend -- --template react-ts
cd unimentee-frontend
npm install

# Core dependencies
npm install react-router-dom @tanstack/react-query @tanstack/react-table
npm install zustand axios react-hook-form zod @hookform/resolvers
npm install recharts react-dropzone
npm install lucide-react clsx tailwind-merge

# UI: shadcn/ui setup
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
npm install @radix-ui/react-dialog @radix-ui/react-toast
npm install @radix-ui/react-select @radix-ui/react-badge

# Dev tools
npm install -D vitest @testing-library/react @testing-library/user-event
npm install -D msw prettier eslint
```

4.2 — Folder Structure

```
src/
├── app/
│   ├── App.tsx
│   └── providers.tsx          # React Query + Auth providers
├── routes/
│   └── index.tsx              # All route definitions
├── layouts/
│   ├── DashboardLayout.tsx
│   └── AuthLayout.tsx
├── features/
│   ├── auth/
│   │   ├── api/auth.api.ts
│   │   ├── pages/LoginPage.tsx
```

```
│   │   └── schemas/login.schema.ts
│   ├── student/
│   │   ├── api/student.api.ts
│   │   ├── hooks/useStudents.ts
│   │   ├── pages/StudentDashboardPage.tsx
│   │   └── types/student.types.ts
│   ├── mentor/
│   ├── faculty/
│   ├── marks/
│   ├── attendance/
│   ├── mentorship/
│   └── admin/
├── components/
│   ├── shared/
│   │   ├── RoleGate.tsx
│   │   ├── ProtectedRoute.tsx
│   │   ├── DataTable.tsx
│   │   └── NotificationBell.tsx
│   └── ui/                    # shadcn/ui components
├── hooks/
│   └── usePermission.ts
├── stores/
│   ├── authStore.ts
│   └── uiStore.ts
├── services/
│   └── api.ts                 # axios instance + interceptors
├── lib/
│   └── utils.ts
└── types/
    └── global.types.ts
```

4.3 — API Client (services/api.ts)

Central axios instance. All calls go through here — handles JWT injection, token refresh on 401, and university_id header.

```
import axios from 'axios'
import { useAuthStore } from '@/stores/authStore'


export const api = axios.create({
```

```
  baseURL: import.meta.env.VITE_API_URL ?? 'http://localhost:8000',
})

api.interceptors.request.use((config) => {
  const { token, universityId } = useAuthStore.getState()
  if (token) config.headers.Authorization = `Bearer ${token}`
  if (universityId) config.headers['X-University-Id'] = universityId
  return config
})

api.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401 && !error.config._retry) {
      error.config._retry = true
      // Optionally call refresh endpoint here
      useAuthStore.getState().logout()
      window.location.href = '/login'
    }
    return Promise.reject(error)
  }
)
```

4.4 — Auth Store (stores/authStore.ts)

```
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

interface AuthState {
  token: string | null
  user: { user_id: number; full_name: string; email: string } | null
  roles: string[]
  permissions: string[]
  universityId: number | null
  login: (token: string, user: any, roles: string[], permissions: string[]) => void
  logout: () => void
}
```

```
export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      token: null,
      user: null,
      roles: [],
      permissions: [],
      universityId: null,
      login: (token, user, roles, permissions) =>
        set({ token, user, roles, permissions, universityId: user.university_id }),
      logout: () => set({ token: null, user: null, roles: [], permissions: [],
universityId: null }),
    }),
    { name: 'auth-store' }
  )
)
```

4.5 — UI Store (stores/uiStore.ts)

```
import { create } from 'zustand'


interface UiState {
  sidebarCollapsed: boolean
  activeTermId: number | null
  activeAcademicYearId: number | null
  toggleSidebar: () => void
  setActiveTerm: (termId: number, yearId: number) => void
}


export const useUiStore = create<UiState>((set) => ({
  sidebarCollapsed: false,
  activeTermId: null,
  activeAcademicYearId: null,
  toggleSidebar: () => set((s) => ({ sidebarCollapsed: !s.sidebarCollapsed })),
  setActiveTerm: (termId, yearId) => set({ activeTermId: termId, activeAcademicYearId:
yearId }),
}))
```

4.6 — Permission Hook (hooks/usePermission.ts)

```
import { useAuthStore } from '@/stores/authStore'

export function usePermission(key: string | string[]): boolean {
  const permissions = useAuthStore((s) => s.permissions)
  if (Array.isArray(key)) return key.every((k) => permissions.includes(k))
  return permissions.includes(key)
}
```

4.7 — RoleGate Component (components/shared/RoleGate.tsx)

```
import { usePermission } from '@/hooks/usePermission'
import { ReactNode } from 'react'

interface Props {
  permission: string | string[]
  fallback?: ReactNode
  children: ReactNode
}

export function RoleGate({ permission, fallback = null, children }: Props) {
  const allowed = usePermission(permission)
  return allowed ? <>{children}</> : <>{fallback}</>
}
```

4.8 — ProtectedRoute (components/shared/ProtectedRoute.tsx)

```
import { Navigate, useLocation } from 'react-router-dom'
import { useAuthStore } from '@/stores/authStore'
import { ReactNode } from 'react'

interface Props {
  children: ReactNode
  requiredPermission?: string
}

export function ProtectedRoute({ children, requiredPermission }: Props) {
  const { token, permissions } = useAuthStore()
```

```
  const location = useLocation()


  if (!token) return <Navigate to='/login' state={{ from: location }} replace />


  if (requiredPermission && !permissions.includes(requiredPermission)) {
    return <div className='p-8 text-red-600'>Access Denied: Missing
{requiredPermission}</div>
  }


  return <>{children}</>
}
```

4.9 — Login Page (features/auth/pages/LoginPage.tsx)

```
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { z } from 'zod'
import { useMutation } from '@tanstack/react-query'
import { useNavigate } from 'react-router-dom'
import { api } from '@/services/api'
import { useAuthStore } from '@/stores/authStore'

const schema = z.object({
  email: z.string().email(),
  password: z.string().min(6),
})
type LoginForm = z.infer<typeof schema>

export function LoginPage() {
  const navigate = useNavigate()
  const login = useAuthStore((s) => s.login)
  const { register, handleSubmit, formState: { errors } } = useForm<LoginForm>({
    resolver: zodResolver(schema),
  })

  const mutation = useMutation({
    mutationFn: async (data: LoginForm) => {
      const res = await api.post('/auth/login', data)
      return res.data
    },
    onSuccess: async (data) => {
      // Fetch user profile after login
      const meRes = await api.get('/auth/me', {
        headers: { Authorization: `Bearer ${data.access_token}` }
      })
      const { user, roles, permissions } = meRes.data
      login(data.access_token, user, roles, permissions)
      navigate('/dashboard')
    },
  })
```

```
  return (
    <div className='min-h-screen flex items-center justify-center bg-gray-50'>
      <div className='bg-white p-8 rounded-xl shadow-md w-full max-w-md'>
        <h1 className='text-2xl font-bold text-blue-700 mb-6'>UniMentee ERP</h1>
        <form onSubmit={handleSubmit((d) => mutation.mutate(d))} className='space-y-4'>
          <div>
            <label className='text-sm font-medium text-gray-700'>Email</label>
            <input {...register('email')}
              className='mt-1 w-full border border-gray-300 rounded-lg px-3 py-2 text-
sm'
              placeholder='you@rvu.edu.in' />
            {errors.email && <p className='text-red-500 text-xs mt-
1'>{errors.email.message}</p>}
          </div>
          <div>
            <label className='text-sm font-medium text-gray-700'>Password</label>
            <input type='password' {...register('password')}
              className='mt-1 w-full border border-gray-300 rounded-lg px-3 py-2 text-
sm' />
            {errors.password && <p className='text-red-500 text-xs mt-
1'>{errors.password.message}</p>}
          </div>
          <button type='submit'
            className='w-full bg-blue-700 text-white py-2 rounded-lg font-semibold
hover:bg-blue-800'
            disabled={mutation.isPending}>
            {mutation.isPending ? 'Signing in...' : 'Sign In'}
          </button>
          {mutation.isError && <p className='text-red-500 text-sm text-center'>Invalid
email or password</p>}
        </form>
      </div>
    </div>
  )
}
```

4.10 — Dashboard Layout (layouts/DashboardLayout.tsx)

```
import { Outlet, NavLink, useNavigate } from 'react-router-dom'
import { useAuthStore } from '@/stores/authStore'
import { useUiStore } from '@/stores/uiStore'
```

```
import { Home, Users, BookOpen, ClipboardList, MessageSquare,
         BarChart2, Settings, LogOut, ChevronLeft } from 'lucide-react'


const navByRole: Record<string, { to: string; icon: any; label: string }[]> = {
  STUDENT: [
    { to: '/student/dashboard', icon: Home,          label: 'Dashboard' },
    { to: '/student/subjects',  icon: BookOpen,      label: 'My Subjects' },
    { to: '/student/attendance',icon: ClipboardList, label: 'Attendance' },
    { to: '/student/performance', icon: BarChart2,   label: 'Performance' },
    { to: '/student/mentor-notes', icon: MessageSquare, label: 'Mentor Notes' },
  ],
  FACULTY: [
    { to: '/faculty/dashboard', icon: Home,          label: 'Dashboard' },
    { to: '/faculty/subjects',  icon: BookOpen,      label: 'Subjects' },
    { to: '/faculty/timetable', icon: ClipboardList, label: 'Timetable' },
    { to: '/faculty/workload',  icon: BarChart2,     label: 'Workload' },
  ],
  FACULTY_MENTOR: [  // mentor role uses FACULTY role with extra nav
    { to: '/mentor/dashboard',  icon: Home,          label: 'Dashboard' },
    { to: '/mentor/mentees',    icon: Users,         label: 'My Mentees' },
    { to: '/mentor/sessions',   icon: MessageSquare, label: 'Sessions' },
    { to: '/mentor/reports',    icon: BarChart2,     label: 'Reports' },
  ],
  ADMIN: [
    { to: '/admin/dashboard',   icon: Home,          label: 'Dashboard' },
    { to: '/admin/users',       icon: Users,         label: 'Users' },
    { to: '/admin/programs',    icon: BookOpen,      label: 'Programs' },
    { to: '/admin/students',    icon: Users,         label: 'Students' },
    { to: '/admin/analytics',   icon: BarChart2,     label: 'Analytics' },
    { to: '/admin/audit-log',   icon: ClipboardList, label: 'Audit Log' },
    { to: '/admin/settings',    icon: Settings,      label: 'Settings' },
  ],
}


export function DashboardLayout() {
  const { user, roles, logout } = useAuthStore()
  const { sidebarCollapsed, toggleSidebar } = useUiStore()
  const navigate = useNavigate()
```

```
  // Determine nav items from primary role
  const primaryRole = roles[0] ?? 'STUDENT'
  const navItems = navByRole[primaryRole] ?? navByRole.STUDENT


  return (
    <div className='flex h-screen bg-gray-100'>
      {/* Sidebar */}
      <aside className={`${sidebarCollapsed ? 'w-16' : 'w-60'} bg-blue-800 text-white
        flex flex-col transition-all duration-200`}>
        <div className='p-4 flex items-center justify-between'>
          {!sidebarCollapsed && <span className='font-bold text-lg'>UniMentee</span>}
          <button onClick={toggleSidebar} className='p-1 hover:bg-blue-700 rounded'>
            <ChevronLeft className={sidebarCollapsed ? 'rotate-180' : ''} size={18} />
          </button>
        </div>
        <nav className='flex-1 px-2 space-y-1'>
          {navItems.map(({ to, icon: Icon, label }) => (
            <NavLink key={to} to={to}
              className={({ isActive }) =>
                `flex items-center gap-3 px-3 py-2 rounded-lg text-sm
                 ${isActive ? 'bg-blue-600 font-semibold' : 'hover:bg-blue-700'}`}
            >
              <Icon size={18} />
              {!sidebarCollapsed && label}
            </NavLink>
          ))}
        </nav>
        <div className='p-4 border-t border-blue-700'>
          {!sidebarCollapsed && (
            <p className='text-xs text-blue-200 mb-2'>{user?.full_name}</p>
          )}
          <button onClick={() => { logout(); navigate('/login') }}
            className='flex items-center gap-2 text-sm hover:text-red-300'>
            <LogOut size={16} /> {!sidebarCollapsed && 'Sign out'}
          </button>
        </div>
      </aside>
      {/* Main */}
```

```
      <main className='flex-1 overflow-auto'>
        <Outlet />
      </main>
    </div>
  )
}
```

4.11 — Route Architecture (routes/index.tsx)

```
import { createBrowserRouter, RouterProvider } from 'react-router-dom'

import { lazy, Suspense } from 'react'

import { DashboardLayout } from '@/layouts/DashboardLayout'

import { ProtectedRoute } from '@/components/shared/ProtectedRoute'

import { LoginPage } from '@/features/auth/pages/LoginPage'



// Lazy-load workspaces (code-split by role)

const StudentDashboard   = lazy(() =>
import('@/features/student/pages/StudentDashboardPage'))

const StudentPerformance = lazy(() =>
import('@/features/student/pages/StudentPerformancePage'))

const StudentAttendance  = lazy(() =>
import('@/features/student/pages/StudentAttendancePage'))

const MentorDashboard    = lazy(() =>
import('@/features/mentor/pages/MentorDashboardPage'))

const MenteesListPage    = lazy(() =>
import('@/features/mentor/pages/MenteesListPage'))

const MenteeDetailPage   = lazy(() =>
import('@/features/mentor/pages/MenteeDetailPage'))

const FacultyDashboard   = lazy(() =>
import('@/features/faculty/pages/FacultyDashboardPage'))

const AttendanceMark     = lazy(() =>
import('@/features/attendance/pages/AttendanceMarkPage'))

const MarksEntry         = lazy(() => import('@/features/marks/pages/MarksEntryPage'))

const AdminDashboard     = lazy(() =>
import('@/features/admin/pages/AdminDashboardPage'))



const Loading = () => <div className='p-8 text-gray-400'>Loading...</div>



export const router = createBrowserRouter([

  { path: '/login', element: <LoginPage /> },

  {

    path: '/',

    element: <ProtectedRoute><DashboardLayout /></ProtectedRoute>,

    children: [

      // Student workspace

      { path: 'student/dashboard',

        element: <Suspense fallback={<Loading />}><StudentDashboard /></Suspense> },

      { path: 'student/performance',

        element: <Suspense fallback={<Loading />}><StudentPerformance /></Suspense> },

      { path: 'student/attendance',
```

```
        element: <Suspense fallback={<Loading />}><StudentAttendance /></Suspense> },
      // Mentor workspace
      { path: 'mentor/dashboard',
        element: <Suspense fallback={<Loading />}><MentorDashboard /></Suspense> },
      { path: 'mentor/mentees',
        element: <Suspense fallback={<Loading />}><MenteesListPage /></Suspense> },
      { path: 'mentor/mentees/:studentId',
        element: <Suspense fallback={<Loading />}><MenteeDetailPage /></Suspense> },
      // Faculty workspace
      { path: 'faculty/dashboard',
        element: <Suspense fallback={<Loading />}><FacultyDashboard /></Suspense> },
      { path: 'faculty/subjects/:offeringId/attendance',
        element: <Suspense fallback={<Loading />}><AttendanceMark /></Suspense> },
      { path: 'faculty/subjects/:offeringId/marks',
        element: <Suspense fallback={<Loading />}><MarksEntry /></Suspense> },
      // Admin workspace
      { path: 'admin/dashboard',
        element: <ProtectedRoute requiredPermission='ORG_VIEW'>
          <Suspense fallback={<Loading />}><AdminDashboard /></Suspense>
        </ProtectedRoute> },
    ]
  }
])

export function AppRouter() {
  return <RouterProvider router={router} />
}
```

4.12 — Student Dashboard Page

```
// features/student/pages/StudentDashboardPage.tsx

import { useQuery } from '@tanstack/react-query'

import { api } from '@/services/api'

import { useAuthStore } from '@/stores/authStore'


export default function StudentDashboardPage() {

  const user = useAuthStore((s) => s.user)


  const { data: attendance } = useQuery({

    queryKey: ['attendance', 'summary', user?.user_id],

    queryFn: () => api.get(`/students/${user?.user_id}/attendance-summary`).then(r =>
r.data),

    enabled: !!user?.user_id,

    staleTime: 30_000,

  })


  const { data: progress } = useQuery({

    queryKey: ['academic', 'progress', user?.user_id],

    queryFn: () => api.get(`/students/${user?.user_id}/progress`).then(r => r.data),

    enabled: !!user?.user_id,

  })


  const { data: sessions } = useQuery({

    queryKey: ['mentor', 'sessions', user?.user_id],

    queryFn: () => api.get('/mentor/my-sessions').then(r => r.data),

    enabled: !!user?.user_id,

  })


  return (

    <div className='p-6 space-y-6'>

      <h1 className='text-2xl font-bold text-gray-800'>

        Welcome, {user?.full_name}

      </h1>

      <div className='grid grid-cols-1 md:grid-cols-3 gap-4'>

        {/* CGPA Card */}

        <div className='bg-white rounded-xl p-5 shadow-sm border border-gray-100'>

          <p className='text-sm text-gray-500'>CGPA</p>
```

```
          <p className='text-3xl font-bold text-blue-700'>{progress?.cgpa ?? '—'}</p>
        </div>
        {/* Overall Attendance */}
        <div className='bg-white rounded-xl p-5 shadow-sm border border-gray-100'>
          <p className='text-sm text-gray-500'>Overall Attendance</p>
          <p className={`text-3xl font-bold ${
            (attendance?.avg_pct ?? 100) < 75 ? 'text-red-600' : 'text-green-600'}`}>
            {attendance?.avg_pct ? `${attendance.avg_pct}%` : '—'}
          </p>
        </div>
        {/* Mentor Sessions */}
        <div className='bg-white rounded-xl p-5 shadow-sm border border-gray-100'>
          <p className='text-sm text-gray-500'>Mentor Sessions</p>
          <p className='text-3xl font-bold text-purple-700'>{sessions?.length ?? 0}</p>
        </div>
      </div>
    </div>
  )
}
```

4.13 — Mentor Dashboard Page

```
// features/mentor/pages/MentorDashboardPage.tsx
import { useQuery } from '@tanstack/react-query'
import { api } from '@/services/api'
import { useAuthStore } from '@/stores/authStore'
import { Link } from 'react-router-dom'
import { AlertTriangle } from 'lucide-react'

export default function MentorDashboardPage() {
  const user = useAuthStore((s) => s.user)

  const { data: assignments = [] } = useQuery({
    queryKey: ['mentor', 'assignments'],
    queryFn: () => api.get('/mentor/assignments').then(r => r.data),
  })

  // At-risk: fetch from analytics endpoint
```

```
  const { data: atRisk = [] } = useQuery({
    queryKey: ['analytics', 'at-risk', user?.user_id],
    queryFn: () => api.get(`/analytics/at-risk?mentorId=${user?.user_id}`).then(r =>
r.data),
    staleTime: 60_000,
  })

  return (
    <div className='p-6 space-y-6'>
      <h1 className='text-2xl font-bold text-gray-800'>Mentor Dashboard</h1>
      <div className='grid grid-cols-3 gap-4'>
        <StatCard label='Total Mentees' value={assignments.length} color='blue' />
        <StatCard label='At-Risk Students' value={atRisk.length} color='red' />
      </div>
      {atRisk.length > 0 && (
        <div className='bg-red-50 border border-red-200 rounded-xl p-4'>
          <h2 className='font-semibold text-red-700 flex items-center gap-2 mb-3'>
            <AlertTriangle size={18} /> Students Needing Attention
          </h2>
          <div className='space-y-2'>
            {atRisk.map((s: any) => (
              <Link key={s.student_id} to={`/mentor/mentees/${s.student_id}`}
                className='flex justify-between items-center bg-white rounded-lg px-4
py-2
                  hover:bg-red-50 border border-red-100'>
                <span className='font-medium text-gray-800'>{s.full_name}</span>
                <span className='text-sm text-red-600'>
                  {s.attendance_pct < 75 ? `Attendance: ${s.attendance_pct}%` : `CGPA:
${s.cgpa}`}
                </span>
              </Link>
            ))}
          </div>
        </div>
      )}
    </div>
  )
}
```

```
function StatCard({ label, value, color }: any) {
  const c = color === 'red' ? 'text-red-600' : 'text-blue-700'
  return (
    <div className='bg-white rounded-xl p-5 shadow-sm border border-gray-100'>
      <p className='text-sm text-gray-500'>{label}</p>
      <p className={`text-3xl font-bold ${c}`}>{value}</p>
    </div>
  )
}
```

4.14 — Attendance Marking Page (Faculty)

```
// features/attendance/pages/AttendanceMarkPage.tsx
import { useState } from 'react'
import { useParams } from 'react-router-dom'
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { api } from '@/services/api'

type Status = 'PRESENT' | 'ABSENT' | 'LATE'

export default function AttendanceMarkPage() {
  const { offeringId } = useParams()
  const qc = useQueryClient()
  const [records, setRecords] = useState<Record<number, Status>>({})

  // Load enrolled students for this offering
  const { data: students = [] } = useQuery({
    queryKey: ['students', 'enrolled', offeringId],
    queryFn: () => api.get(`/offerings/${offeringId}/enrolled-students`).then(r =>
r.data),
  })

  // Load today's session or create one
  const { data: session } = useQuery({
    queryKey: ['attendance', 'session', offeringId, 'today'],
    queryFn: () => api.get(`/offerings/${offeringId}/sessions?date=today`).then(r =>
r.data[0]),
  })
```

```
  // Bulk submit mutation
  const saveMutation = useMutation({
    mutationFn: (sessionId: number) =>
      api.put(`/sessions/${sessionId}/attendance`, {
        records: Object.entries(records).map(([sid, status]) => ({
          student_id: Number(sid), status
        }))
      }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['attendance'] }),
  })

  const toggle = (studentId: number, status: Status) =>
    setRecords(prev => ({ ...prev, [studentId]: status }))

  return (
    <div className='p-6'>
      <h1 className='text-xl font-bold mb-4'>Mark Attendance</h1>
      {session?.is_locked && (
        <div className='bg-amber-50 border border-amber-200 rounded-lg p-3 mb-4 text-
amber-800'>
          Session is locked. Contact admin to edit.
        </div>
      )}
      <div className='bg-white rounded-xl border'>
        <table className='w-full text-sm'>
          <thead className='bg-gray-50'>
            <tr>
              <th className='text-left px-4 py-3 font-medium text-gray-
600'>Student</th>
              <th className='text-left px-4 py-3 font-medium text-gray-600'>Status</th>
            </tr>
          </thead>
          <tbody>
            {students.map((s: any) => (
              <tr key={s.student_id} className='border-t'>
                <td className='px-4 py-3 font-medium'>{s.full_name} — {s.usn}</td>
                <td className='px-4 py-3'>
                  <div className='flex gap-2'>
                    {(['PRESENT','ABSENT','LATE'] as Status[]).map(st => (
```

```
                    <button key={st}
                      onClick={() => toggle(s.student_id, st)}
                      disabled={session?.is_locked}
                      className={`px-3 py-1 rounded-full text-xs font-semibold
                        ${ records[s.student_id] === st
                          ? st === 'PRESENT' ? 'bg-green-500 text-white'
                          : st === 'LATE' ? 'bg-amber-500 text-white'
                          : 'bg-red-500 text-white'
                        : 'bg-gray-100 text-gray-600'}`}
                    >{st}</button>
                  ))}
                </div>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
    <button onClick={() => session && saveMutation.mutate(session.session_id)}
      disabled={!session || session.is_locked}
      className='mt-4 bg-blue-700 text-white px-6 py-2 rounded-lg font-semibold
hover:bg-blue-800'>
      Save Attendance
    </button>
  </div>
 )
}
```

4.15 — Marks Entry Page (with Optimistic Updates)

```
// features/marks/pages/MarksEntryPage.tsx
import { useParams } from 'react-router-dom'
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { useState } from 'react'
import { api } from '@/services/api'
import toast from 'react-hot-toast'

export default function MarksEntryPage() {
  const { offeringId } = useParams()
  const qc = useQueryClient()

  const { data: assessments = [] } = useQuery({
    queryKey: ['marks', 'assessments', offeringId],
    queryFn: () => api.get(`/offerings/${offeringId}/assessments`).then(r => r.data),
  })

  const [selectedAssessment, setSelectedAssessment] = useState<any>(null)

  const { data: marks = [] } = useQuery({
    queryKey: ['marks', 'list', selectedAssessment?.assessment_id],
    queryFn: () =>
api.get(`/assessments/${selectedAssessment.assessment_id}/marks`).then(r => r.data),
    enabled: !!selectedAssessment,
  })

  const markMutation = useMutation({
    mutationFn: ({ assessmentId, studentId, marksObtained, version }: any) =>
      api.put(`/assessments/${assessmentId}/marks/${studentId}`, {
        marks_obtained: marksObtained, is_absent: false, version
      }).then(r => r.data),
    onMutate: async (vars) => {
      const key = ['marks', 'list', vars.assessmentId]
      await qc.cancelQueries({ queryKey: key })
      const prev = qc.getQueryData(key)
      qc.setQueryData(key, (old: any[]) =>
        old?.map(m => m.student_id === vars.studentId
          ? { ...m, marks_obtained: vars.marksObtained } : m))
```

```
      return { prev, key }
    },
    onError: (err: any, _, ctx) => {
      qc.setQueryData(ctx!.key, ctx!.prev)
      if (err.response?.status === 409) toast.error('Version conflict — refresh and
retry')
      else toast.error(err.response?.data?.detail ?? 'Error saving mark')
    },
    onSettled: (_, __, vars) =>
      qc.invalidateQueries({ queryKey: ['marks', 'list', vars.assessmentId] }),
  })


  return (
    <div className='p-6 space-y-4'>
      <h1 className='text-xl font-bold'>Marks Entry</h1>
      <div className='flex gap-2 flex-wrap'>
        {assessments.map((a: any) => (
          <button key={a.assessment_id}
            onClick={() => setSelectedAssessment(a)}
            className={`px-4 py-2 rounded-lg text-sm font-medium border
              ${selectedAssessment?.assessment_id === a.assessment_id
                ? 'bg-blue-700 text-white border-blue-700'
                : 'bg-white text-gray-700 border-gray-300 hover:border-blue-400'}`}
          >
            {a.title} / {a.max_marks} marks
          </button>
        ))}
      </div>
      {selectedAssessment && (
        <div className='bg-white rounded-xl border overflow-hidden'>
          <table className='w-full text-sm'>
            <thead className='bg-gray-50'>
              <tr>
                <th className='text-left px-4 py-3'>Student</th>
                <th className='text-left px-4 py-3'>Marks (max
{selectedAssessment.max_marks})</th>
              </tr>
            </thead>
            <tbody>
```

```
              {marks.map((m: any) => (
                <tr key={m.student_id} className='border-t'>
                  <td className='px-4 py-2'>{m.student_id}</td>
                  <td className='px-4 py-2'>
                    <input
                      type='number'
                      min={0} max={selectedAssessment.max_marks}
                      defaultValue={m.marks_obtained ?? ''}
                      className='border border-gray-200 rounded px-2 py-1 w-24 text-sm'
                      onBlur={(e) => {
                        const val = parseFloat(e.target.value)
                        if (!isNaN(val))
                          markMutation.mutate({
                            assessmentId: selectedAssessment.assessment_id,
                            studentId: m.student_id,
                            marksObtained: val,
                            version: m.version,
                          })
                      }}
                    />
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </div>
  )
}
```

4.16 — App Providers and Entry Point

```
// app/providers.tsx
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { ReactNode } from 'react'

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 30_000,      // 30 seconds
      retry: 1,
      refetchOnWindowFocus: false,
    }
  }
})

export function Providers({ children }: { children: ReactNode }) {
  return (
    <QueryClientProvider client={queryClient}>
      {children}
    </QueryClientProvider>
  )
}

// app/App.tsx
import { AppRouter } from '@/routes'
import { Providers } from './providers'

export default function App() {
  return (
    <Providers>
      <AppRouter />
    </Providers>
  )
}

// main.tsx
import React from 'react'
import ReactDOM from 'react-dom/client'
```

```
import App from './app/App'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

4.17 — tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: { extend: {} },
  plugins: [],
}
```

4.18 — Environment Variables

```
# .env (frontend)
VITE_API_URL=http://localhost:8000

# .env (backend)
DATABASE_URL=postgresql://postgres:<password>@db.<project>.supabase.co:5432/postgres
JWT_SECRET=your-secret-key-min-32-chars
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_SERVICE_KEY=your-service-role-key
```

4.19 — Backend: /auth/me Endpoint (Required for Login)

The frontend calls /auth/me after login to get the user's roles and permissions. Add this to your auth router.

```
# In app/routers/auth.py — add to existing file

from app.models.users import User, UserRole, Role, RolePermission, Permission


@router.get('/me')

def get_me(user=Depends(get_current_user), db: Session = Depends(get_db)):

    u = db.query(User).filter(User.user_id == user['user_id']).first()

    if not u: raise HTTPException(404, 'User not found')

    # Get roles

    role_rows = db.query(Role).join(UserRole).filter(

        UserRole.user_id == u.user_id).all()

    roles = [r.name for r in role_rows]

    # Get permissions via roles

    perm_rows =
db.query(Permission).join(RolePermission).join(Role).join(UserRole).filter(

        UserRole.user_id == u.user_id).all()

    permissions = list(set(p.key for p in perm_rows))

    return {

        'user': {

            'user_id': u.user_id,

            'full_name': u.full_name,

            'email': str(u.email),

            'university_id': u.university_id,

        },

        'roles': roles,

        'permissions': permissions,

    }
```

Add these User models if not yet in models/users.py

class UserRole(Base): __tablename__ = 'user_roles' user_role_id = Column(BigInteger, primary_key=True) user_id = Column(BigInteger, ForeignKey('users.user_id')) role_id = Column(BigInteger, ForeignKey('roles.role_id')) class Role(Base): __tablename__ = 'roles' role_id = Column(BigInteger, primary_key=True) name = Column(String(100)) class Permission(Base): __tablename__ = 'permissions' permission_id = Column(BigInteger, primary_key=True) key = Column(String(100)) class RolePermission(Base): __tablename__ = 'role_permissions' role_permission_id = Column(BigInteger, primary_key=True) role_id = Column(BigInteger, ForeignKey('roles.role_id')) permission_id = Column(BigInteger, ForeignKey('permissions.permission_id'))

Section 5 — Testing Strategy

5.1 — Backend: Test login flow with curl

```
# 1. Login
curl -X POST http://localhost:8000/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email": "admin@rvu.edu.in", "password": "hashed_password"}'

# Expected: {"access_token": "eyJ...", "token_type": "bearer"}

# 2. Get /me
curl http://localhost:8000/auth/me \
  -H 'Authorization: Bearer <token>'

# 3. List programs
curl http://localhost:8000/academic/programs \
  -H 'Authorization: Bearer <token>'

# 4. List students in batch
curl 'http://localhost:8000/students?batch_id=1' \
  -H 'Authorization: Bearer <token>'

# 5. Create attendance session
curl -X POST http://localhost:8000/offerings/1/sessions \
  -H 'Authorization: Bearer <token>' \
  -H 'Content-Type: application/json' \
  -d '{"session_date":"2026-03-05","start_time":"10:00:00","end_time":"11:00:00"}'
```

5.2 — Frontend: Component Test Example

```
// features/auth/__tests__/LoginPage.test.tsx
import { render, screen, fireEvent, waitFor } from '@testing-library/react'
import { LoginPage } from '../pages/LoginPage'
import { MemoryRouter } from 'react-router-dom'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { rest } from 'msw'
import { setupServer } from 'msw/node'
```

```
const server = setupServer(
  rest.post('/auth/login', (req, res, ctx) => {
    return res(ctx.json({ access_token: 'test-token', token_type: 'bearer' }))
  }),
  rest.get('/auth/me', (req, res, ctx) => {
    return res(ctx.json({
      user: { user_id: 1, full_name: 'Test User', email: 'test@rvu.edu.in',
university_id: 1 },
      roles: ['STUDENT'], permissions: ['ATTENDANCE_VIEW_OWN', 'MARKS_VIEW_OWN']
    }))
  })
)

beforeAll(() => server.listen())
afterAll(() => server.close())

test('login form submits and redirects', async () => {
  const qc = new QueryClient()
  render(
    <QueryClientProvider client={qc}>
      <MemoryRouter><LoginPage /></MemoryRouter>
    </QueryClientProvider>
  )
  fireEvent.change(screen.getByPlaceholderText(/rvu.edu.in/), { target: { value:
'test@rvu.edu.in' } })
  fireEvent.change(screen.getByLabelText(/password/i), { target: { value: 'password123'
} })
  fireEvent.click(screen.getByText('Sign In'))
  await waitFor(() => expect(screen.queryByText('Signing in...')).toBeNull())
})
```

Section 6 — Execution Checklist

Follow this order exactly. Do not move to the next item until the current one works end-to-end.

#	Task	Done When
1	Stage 4: Add academic models, repos, service, router. Register in main.py	GET /academic/programs returns list
2	Stage 4: Add offering status transitions with version check	PATCH /offerings/1/status returns 409 on version mismatch
3	Stage 5: Add student models + transactional enrollment	POST /students/1/enrollments returns 409 on duplicate
4	Stage 6: Mentor assignments + sessions	GET /mentor/assignments returns active assignments
5	Stage 7: Attendance sessions + bulk records + auto-lock	PUT /sessions/1/attendance saves records; locked session returns 403
6	Stage 8: Assessments + marks + SGPA trigger	PUT /assessments/1/marks/2 returns 400 if marks > max_marks
7	Stage 9: Portfolio upload URL + item creation	POST /portfolio/upload-url returns presigned URL
8	Stage 9-10: Wire audit_service into marks + attendance mutations	audit_logs grows on each mark/status change
9	Add /auth/me endpoint with roles + permissions	Returns {user, roles, permissions} correctly
10	Frontend: npm create vite, install deps, configure tailwind	npm run dev shows blank page with no errors
11	Frontend: authStore, uiStore, api.ts, usePermission hook	TypeScript compiles cleanly
12	Frontend: LoginPage + /auth/login + /auth/me integration	Login with admin@rvu.edu.in stores token + permissions
13	Frontend: DashboardLayout + routes/index.tsx skeleton	Authenticated user sees sidebar after login
14	Frontend: Student dashboard with 3 stat cards	Cards show CGPA, attendance %, mentor session count
15	Frontend: Mentor dashboard with at-risk panel	At-risk list links to /mentor/mentees/:id
16	Frontend: Faculty attendance marking page	Toggle PRESENT/ABSENT, save, lock session
17	Frontend: Marks entry with optimistic updates	Entering marks updates table instantly; 409 shows toast
18	Frontend: RoleGate applied to all sensitive buttons	Student cannot see Verify/Publish buttons
19	End-to-end: Login → mark attendance → save → lock	Complete flow works without errors
20	End-to-end: Login → enter marks → submit → verify → publish	Assessment moves through all status stages

Section 7 — Common Pitfalls & Solutions

Problem	Cause	Fix
Login returns 401 immediately	password_hash in DB is literal 'hashed_password' from seed, not bcrypt	For dev, add a bypass: if password == 'hashed_password': allow. Or re-hash using passlib.
CORS error in frontend	Backend CORS not allowing localhost:5173	Confirm CORSMiddleware in main.py, allow_origins includes Vite port
409 on every attendance session	UNIQUE(offering_id, session_date, start_time) violated	Check you're not creating duplicate sessions for same slot; use GET first to see if session exists
Marks trigger fires 'exceeds max_marks'	DB trigger check_marks_max running correctly	This is correct behavior. Client should validate max before sending.
/auth/me returns empty permissions	UserRole / RolePermission models not mapping	Check user_roles and role_permissions tables have data for the user
React Query refetches on every click	staleTime is 0 (default)	Set staleTime: 30_000 in QueryClient defaultOptions
TypeScript error: 'module has no exported member'	Missing default export on lazy-loaded page	Every page component must have: export default function PageName()
Supabase upload URL 403	Using anon key instead of service key	Use SUPABASE_SERVICE_KEY (service_role) in backend, never expose in frontend

Development Login Shortcut

Since the seed script uses 'hashed_password' as a literal string (not bcrypt), modify your auth_service.py verify_password for development: def verify_password(plain, hashed): if hashed == 'hashed_password' and plain == 'hashed_password': return True # dev bypass return pwd_context.verify(plain, hashed) Remove this before production.

Section 8 — Final Architecture Summary

Layer	Technology	Your Files
Database	PostgreSQL 15 on Supabase	Schema v3.1, Seed script (already run)
ORM	SQLAlchemy + Alembic	app/models/*.py
API Framework	FastAPI + Pydantic v2	app/routers/*.py, app/schemas/*.py
Auth	JWT (python-jose) + bcrypt	app/core/security.py, app/core/rbac.py
Business Logic	Pure Python	app/services/*.py
DB Access	Repository pattern	app/repositories/*.py
File Storage	Supabase Storage	app/services/storage_service.py
UI Framework	React 18 + TypeScript	src/features/, src/components/
Server State	TanStack Query v5	features/*/hooks/use*.ts
Client State	Zustand v4	src/stores/authStore.ts, uiStore.ts
Forms	react-hook-form + zod	features/*/schemas/*.schema.ts
Routing	React Router v6	src/routes/index.tsx
Styling	Tailwind CSS + shadcn/ui	tailwind.config.js, src/components/ui/

This document is self-contained. Build stage by stage, test each before moving to the next, and do not skip the layer architecture. Good luck!